

ONLINE INTERVIEW SCHEDULING SYSTEM

Backend Case Study and Project Report

A REST API for recruiter-managed interview availability and conflict-safe candidate booking

PROJECT AREA	DETAIL
Student	Tanmay Vijay Sherkar
Roll.no:	150096725139
Cohort	Sam Altman
Subject	Backend Development
Submission type	Final project case study

Prepared for academic submission

This report documents the design, implementation, validation, and deployment plan for the Interviewly online interview scheduling system. It matches the accompanying project implementation.

Executive Summary

Interview scheduling is a coordination problem with two distinct sources of conflict: a candidate must not reserve more than one overlapping interview, and a recruiter must not publish overlapping interview windows. Interviewly addresses this problem through a role-aware REST API backed by MongoDB and a React interface that exposes the same business rules to users.

The system models users and interview slots as referenced MongoDB collections. Recruiters create and manage availability. Candidates browse future available slots and book a slot using an authenticated endpoint. JWT middleware identifies the caller, role middleware limits recruiter-only operations, request validation rejects malformed scheduling data, and overlap queries prevent conflicting schedules. The booking endpoint also performs an atomic availability claim inside a MongoDB transaction so two candidates cannot successfully reserve the same slot.

The implementation delivers the requested backend subject outcomes: secure authentication, role-based authorization, referenced schemas, validation, conflict detection, a documented REST API, environment configuration, and a deployment-ready structure. The React client demonstrates the core workflows without moving business logic out of the backend.

Key conclusion: the project demonstrates how a small REST service can turn a real-world scheduling rule into enforceable data, authorization, and transaction boundaries rather than relying on frontend checks alone.

Project Outcomes

OUTCOME	IMPLEMENTATION EVIDENCE
Recruiters publish availability	POST /api/slots, restricted to recruiter accounts.
Candidates book genuinely free slots	POST /api/slots/:id/book uses status checks and a transaction.
No recruiter overlap	Slot creation and updates query overlapping active slots for the recruiter.
No candidate overlap	Booking checks the candidate's existing booked intervals.
Unauthorized slot changes blocked	JWT authentication and recruiter ownership checks protect routes.

Contents

- 1. Introduction and Problem Context
- 2. Requirements Analysis
- 3. System Architecture
- 4. Database Design
- 5. Authentication and Authorization
- 6. Scheduling and Conflict Validation
- 7. REST API Design
- 8. Frontend Integration
- 9. Validation and Error Handling
- 10. Testing Strategy and Results
- 11. Deployment and Configuration
- 12. Limitations and Future Enhancements
- 13. Conclusion
- Appendix A. API Quick Reference
- Appendix B. Project Structure

1 Introduction and Problem Context

Recruitment teams coordinate multiple participants, calendars, interviewers, and time zones. A manual or loosely validated booking process quickly produces duplicate reservations, recruiter conflicts, and unclear ownership of available time. The purpose of this project is to provide a focused scheduling service where availability is published by recruiters and consumed by candidates through a predictable API.

The system is split into a backend service and a basic frontend. The backend is the source of truth for authentication, authorization, validation, and scheduling rules. The frontend provides a practical user interface for demonstrating those workflows, but it does not decide whether a booking is valid.

1.1 Objectives

- Design Candidate, Recruiter, and InterviewSlot data structures.
- Implement JWT authentication for candidates and recruiters.
- Restrict slot publication and modification to authorized recruiters.
- Reject recruiter slots that overlap existing active slots.
- Reject candidate bookings that overlap the candidate's booked interviews.
- Validate required scheduling fields and return clear HTTP errors.
- Provide a Postman or Thunder Client collection for API testing.
- Prepare deployment configuration compatible with MongoDB Atlas and Render or Railway.

1.2 Scope

The implemented scope includes account registration and login, future slot listing, recruiter slot management, candidate booking, ownership enforcement, status transitions, and a responsive React demonstration client. Notifications, calendar-provider synchronization, interviewer assignment beyond the recruiter, and automated email delivery are outside the current scope.

2 Requirements Analysis

The requirements were converted into functional rules that can be tested at the API boundary. A scheduling system is correct only when its rules are enforced independently of the client that sends the request.

REQUIREMENT	ACTOR	RULE	EXPECTED RESULT
Register account	Candidate or recruiter	Valid identity fields and unique email.	201 or validation/conflict error.
Create slot	Recruiter	Start precedes end and interval does not overlap.	Available slot created.
List slots	Any client	Only future slots are returned by default.	Sorted list of available slots.
Book slot	Candidate	Slot is available and candidate has no overlap.	Slot changes to BOOKED.
Edit or cancel slot	Slot owner recruiter	Only available slots can be changed.	Updated slot or cancellation.

2.1 Non-functional expectations

- Security: passwords are hashed and tokens are signed with an environment secret.

- Consistency: booking must not succeed against a slot another request has already claimed.
- Maintainability: routes, models, and middleware are separated by responsibility.
- Deployability: configuration is read from environment variables.
- Usability: responses contain clear status codes and messages.

3 System Architecture

Interviewly follows a three-layer web architecture. The React client sends HTTP requests through Axios. The Express application receives those requests, applies middleware and route-level validation, and invokes Mongoose operations. MongoDB stores users and interview slots as related collections.

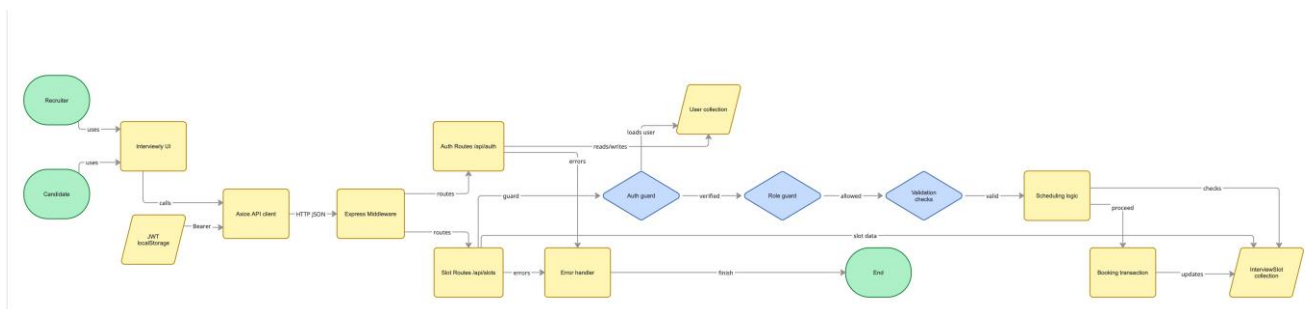
LAYER	RESPONSIBILITY	IMPLEMENTATION
Presentation	Authentication, slot browsing, booking, recruiter management.	React, Vite, Axios, lucide-react.
API	Routing, validation, JWT verification, role and ownership checks.	Node.js, Express.js, express-validator.
Data	Users, slots, references, indexes, status changes, transactions.	MongoDB Atlas and Mongoose.

3.1 Request flow

1. The user signs in or registers. The API validates credentials and returns a signed JWT.
2. The frontend stores the token and sends it as a Bearer token on protected requests.
3. Authentication verifies the token, loads the current user, and attaches the user to the request.
4. Role middleware checks whether the user can perform the requested operation.
5. The route validates input and applies scheduling rules before writing to MongoDB.
6. The API returns a status code and JSON response; the client updates the schedule.

3.2 Architectural decision

The backend retains all business decisions. The frontend may disable a button for convenience, but the server still checks authentication, ownership, status, time ordering, and overlap for every request. This protects the system when requests come from Postman, a different frontend, or a malicious client.



4 Database Design

The database uses two collections. A single User model represents both candidates and recruiters through a role field. InterviewSlot references the recruiter who owns the slot and optionally references the candidate who books it. This keeps user information normalized and allows populated responses for display.

4.1 User schema

FIELD	TYPE	RULE / PURPOSE
name	String	Required, trimmed, minimum two characters.
email	String	Required, lowercased, unique, used for login.
password	String	Hashed with bcrypt and excluded by default.
role	Enum	candidate or recruiter.
createdAt / updatedAt	Date	Automatic audit timestamps.

4.2 InterviewSlot schema

FIELD	TYPE	RULE / PURPOSE
recruiter	ObjectId reference	Required reference to User and schedule owner.
candidate	ObjectId reference	Nullable until a candidate books.
startTime / endTime	Date	Required interval; end must be later.
status	Enum	AVAILABLE, BOOKED, or CANCELLED.
title	String	Human-readable interview name.
meetingLink	String	Optional HTTPS meeting URL.

4.3 Indexing strategy

The slot model indexes recruiter and start time for recruiter schedule queries, candidate and status for candidate schedule queries, and status and start time for future availability listing. These indexes support the conflict queries executed on create and booking operations.

5 Authentication and Authorization

Authentication answers who is making a request. Authorization answers whether that user is allowed to perform the action. Interviewly keeps the two responsibilities separate in middleware.

5.1 Registration and password protection

During registration, the API validates the payload and checks for an existing email address. The User model hashes a new password with bcrypt before saving. The stored password is excluded by default. Login selects the password field only long enough to compare the submitted password.

5.2 JWT workflow

After successful registration or login, the API signs a token containing the user identifier and role. The token expires after seven days. Protected routes require:

```
Authorization: Bearer <jwt-token>
```

The protect middleware verifies the token using JWT_SECRET, loads the user from MongoDB, and attaches the user to req.user. Invalid, expired, missing, or stale tokens result in HTTP 401.

5.3 Role and ownership rules

CHECK	ENFORCEMENT
Recruiter-only creation	roles('recruiter') protects POST /api/slots.
Candidate-only booking	roles('candidate') protects POST /api/slots/:id/book.
Recruiter ownership	Update and cancellation queries include recruiter: req.user._id.
Booked slot protection	Booked interviews cannot be rescheduled through the recruiter update flow.

6 Scheduling and Conflict Validation

Scheduling correctness depends on interval arithmetic. Two intervals overlap when the first starts before the second ends and the first ends after the second starts. The implementation expresses this as:

```
startTime < requestedEndTime AND endTime > requestedStartTime
```

This allows adjacent appointments. A 10:00–10:30 interview and a 10:30–11:00 interview do not overlap.

6.1 Recruiter conflict validation

When a recruiter creates a slot, the API searches for an active slot owned by the same recruiter whose interval overlaps the requested interval. CANCELLED slots are ignored. A conflict returns HTTP 409 and no new slot is created. Updates apply the same rule while excluding the slot being edited.

6.2 Candidate conflict validation

When a candidate books a slot, the API searches for any existing slot assigned to that candidate with a non-cancelled status and an overlapping interval. A conflict returns HTTP 409, preserving the candidate's existing schedule.

6.3 Preventing double booking

The booking endpoint opens a MongoDB session and transaction, finds the requested slot only when its status is AVAILABLE, checks candidate overlap, and updates the slot to BOOKED with the candidate reference. The status condition is part of the write filter as well, so a second request cannot update the same slot after the first request claims it.

6.4 Booking sequence

1. Start a MongoDB session.
2. Find the requested slot with status AVAILABLE.
3. Reject the request if the slot is missing, booked, or cancelled.

4. Search for an overlapping booked slot belonging to the candidate.
5. Update the slot atomically: set candidate and status BOOKED.
6. Commit the transaction and return the booked slot.

7 REST API Design

The API is organized under the /api prefix. JSON is used for request and response bodies. Protected operations use the Bearer token established during login.

METHOD	ENDPOINT	ACCESS	PURPOSE
GET	/api/health	Public	Service health check.
POST	/api/auth/register	Public	Create candidate or recruiter account.
POST	/api/auth/login	Public	Authenticate and return JWT.
GET	/api/auth/me	Authenticated	Return current user.
GET	/api/slots	Public	List future available slots.
GET	/api/slots/mine	Authenticated	List personal schedule.
POST	/api/slots	Recruiter	Publish an available slot.
POST	/api/slots/:id/book	Candidate	Book an available slot.
PATCH	/api/slots/:id	Owner recruiter	Edit an available slot.
DELETE	/api/slots/:id	Owner recruiter	Cancel an available slot.

7.1 Example create-slot request

```
{
  "title": "Frontend interview",
  "startTime": "2027-01-15T10:00:00.000Z",
  "endTime": "2027-01-15T10:45:00.000Z",
  "meetingLink": "https://meet.google.com/example"
}
```

7.2 Status code conventions

STATUS	MEANING
200 OK	Successful read, update, cancellation, or booking.
201 Created	Account or slot created.
401 Unauthorized	Missing, invalid, or expired token.
403 Forbidden	Wrong role or unauthorized action.
404 Not Found	Resource does not exist or is not owned.
409 Conflict	Schedule overlap, duplicate email, or claimed slot.
422 Unprocessable Entity	Invalid field or time data.

8 Frontend Integration

The Vite React application consumes the REST API through Axios. It provides two role-specific experiences while keeping validation and business decisions on the server.

VIEW	CAPABILITIES
------	--------------

Authentication	Sign in, register, choose role, and display API errors.
Candidate dashboard	Search slots, view date and time, book, and view personal schedule.
Recruiter dashboard	Publish, edit, cancel available slots, and view booked candidates.
Shared shell	Token persistence, logout, notices, responsive layout, local-time display.

The client reads the API base URL from `VITE_API_URL`, allowing local and deployed environments to use different backend URLs without changing application code.

9 Validation and Error Handling

Validation is applied at multiple levels. `express-validator` handles field shape and format. Business validation checks relationships and time semantics. `Mongoose` validates schema types and enum values before persistence.

VALIDATION	EXAMPLE
Required identity fields	Name, email, password, and role are required.
Email format	Invalid email values are rejected before authentication.
Password length	Passwords must contain at least six characters.
ISO dates	<code>startTime</code> and <code>endTime</code> must be valid ISO dates.
Time ordering	<code>endTime</code> must be later than <code>startTime</code> .
Meeting link	Optional link must use HTTP or HTTPS.
Schedule conflict	Overlapping intervals return 409.

Errors are returned as JSON with a message field so Postman and the React client can present a readable result. A final Express error handler catches unexpected exceptions and returns a generic server error while logging the original exception on the server.

10 Testing Strategy and Results

Testing is organized around business rules rather than only route availability. The most important tests are the negative cases that demonstrate invalid or conflicting schedules cannot enter the database.

TEST CASE	EXPECTED RESULT	RESULT
Register valid candidate	201 with token and public profile.	Implemented
Register duplicate email	409; second user not created.	Implemented
Login wrong password	401 unauthorized.	Implemented
Create slot as candidate	403 forbidden.	Implemented
End before start	422 validation error.	Implemented
Overlapping recruiter slot	409; no new slot.	Implemented
Book available slot	Slot becomes BOOKED.	Implemented
Book claimed slot	409; second booking rejected.	Implemented
Overlapping candidate booking	409; schedule preserved.	Implemented
Edit another recruiter slot	404; ownership prevents change.	Implemented
React production build	Vite build succeeds.	Passed
API syntax validation	Server modules pass checks.	Passed

10.1 Verification performed

The React client was installed and compiled with the Vite production build. Server modules were checked for syntax errors. The client's first screen was opened in a local browser for a visual smoke test. Full booking integration requires a configured MongoDB instance; the code is prepared for MongoDB Atlas and transactions should be tested against a replica set deployment.

11 Deployment and Configuration

The project can deploy as two services. The Express API can run on Render or Railway, while the React client can run on Vercel or Netlify. MongoDB Atlas provides the production database and replica-set behavior needed for transaction-backed booking.

11.1 Backend environment variables

VARIABLE	PURPOSE	EXAMPLE
PORT	HTTP port used by the API.	5000
MONGO_URI	MongoDB Atlas connection string.	mongodb+srv://.../interviewly
JWT_SECRET	Long random token-signing secret.	Secret manager value
CLIENT_URL	Allowed frontend origin.	https://interviewly.example.com

11.2 Deployment sequence

1. Create a MongoDB Atlas cluster and database user.
2. Create a backend service from the server directory and set environment variables.
3. Use npm install during build and npm start in production.
4. Deploy the React client with VITE_API_URL pointing to the live API.
5. Import the included Postman collection and verify health, authentication, creation, and booking.

12 Limitations and Future Enhancements

The current implementation is deliberately focused. The following improvements would make it more production-complete:

- Refresh tokens or short-lived access tokens with revocation.
- Password reset, email verification, and authentication rate limiting.
- Pagination, date-range filters, recruiter filters, and timezone preferences.
- Candidate cancellation and recruiter rescheduling with audit history.
- A separate Interviewer model for multi-interviewer organizations.
- Automated email or calendar invitations after booking.
- Integration tests using a test MongoDB replica set and CI checks.
- Structured logging, metrics, and monitoring for conflicts and authentication failures.

13 Conclusion

Interviewly meets the core requirements of an online interview scheduling system. The design separates identity, authorization, data modeling, and scheduling rules so each concern can be tested and maintained independently. Recruiters can publish availability without overlapping their own schedule, candidates can reserve only genuinely available time, and concurrent requests are protected by status-aware writes and transaction boundaries.

The project demonstrates a central backend principle: validation performed only in a browser is not a business rule. By placing conflict detection and ownership checks in the Express API, every client receives the same reliable behavior. The result is a practical REST service that can be extended with notifications, calendar synchronization, and richer recruitment workflows.

Appendix A API Quick Reference

Protected requests use:

Authorization: Bearer <token>

ACTION	METHOD AND URL	BODY / QUERY
Register	POST /api/auth/register	name, email, password, role
Login	POST /api/auth/login	email, password
Current user	GET /api/auth/me	Bearer token
Browse slots	GET /api/slots	Optional status query
My schedule	GET /api/slots/mine	Bearer token
Publish slot	POST /api/slots	title, startTime, endTime, meetingLink
Book slot	POST /api/slots/:id/book	Bearer token
Update slot	PATCH /api/slots/:id	Updated scheduling fields
Cancel slot	DELETE /api/slots/:id	Bearer token

Appendix B Project Structure

```

137-online-interview-scheduling-system-problem/
├── README.md
├── client/
│   ├── src/main.jsx
│   ├── src/styles.css
│   ├── package.json
│   └── .env.example
├── server/
│   ├── src/app.js
│   ├── src/server.js
│   ├── src/models/User.js
│   ├── src/models/InterviewSlot.js
│   ├── src/middleware/auth.js
│   ├── src/routes/auth.js
│   ├── src/routes/slots.js
│   ├── package.json
│   └── .env.example
└── postman/
    └── Interviewly.postman_collection.json
  
```

Appendix C Submission Notes

Before a live demonstration, configure server/.env with a reachable MongoDB Atlas URI and JWT secret, start the API, start the client, and use separate recruiter and candidate accounts to demonstrate publishing, booking, and conflict rejection. The backend is the primary subject deliverable; the React client is included to demonstrate the REST API in a realistic workflow.